॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Postiwala Vatsal **PRN: 26070122509**

Semester: III **Year: AY 2026 -2027**

Subject Title: Operating Systems Lab	
EXPERIMENT No: 6	
TITLE: Loop Statements	**DoP**: 17-08-2026

Aim: Implement shell script to demonstrate loop statements.

Learning Outcome: 1. To understand the loop statements
2. To demonstrate shell script to implement loop statements using Linux Commands

Hardware/Software:

This experiment is performed using Ubuntu OS in a virtualized environment within Windows 11. The software used is:

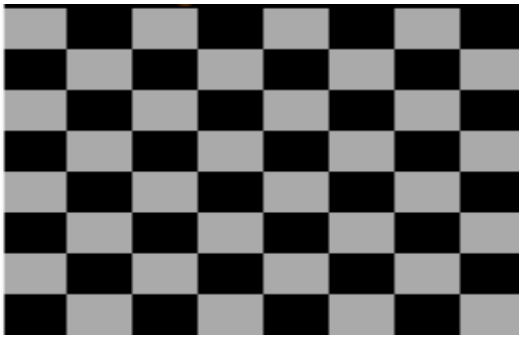
- VMware Workstation 17 Player (Free Virtualization Tool)
- Ubuntu 22.04.1 LTS ISO file
- VMware Tools installed on the virtual machine

Problem Statement:

a) Shell script using for loop to print the pattern: (ask the user to enter the value of n)

```
*
**
***
****
*****
******
*****
****
***
**
*
```

b) Shell script using for loop to display a chessboard on screen. The chessboard should consist of 64 squares - eight rows and eight columns arranged in two alternating colors



Hint:

….

…..

if […..]

then

echo -e -n " \033[47m" //light grey

else

echo -e -n "\033[40m" //black

c) Write a shell script using for loop which considers Day 1 as Mon (weekday) and accordingly considers Day 7 as Sun (weekend) and print the wage as Rs 350 for weekday and Rs 550 for weekend.

Hint: for day in Mon Tue Wed Thu Fri Sat Sun

Output should be:

Day 1: Mon (weekday), wage = 350
Day 2: Tue (weekday), wage = 350
Day 3: Wed (weekday), wage = 350
…..
….
Day 7: Sun (weekend), wage = 550

d) Shell script to print the factorial of a number using for loop

Theory:

The shell acts as an interface between the user and the OS services. It accepts human-readable commands from the user and converts them into a language that the kernel can understand. The shell can be accessed using the terminal.

Shell accepts the command as input from the user and executes it. A shell can also take inputs as a file and execute them in case of executing a bunch of commands routinely. This helps to avoid the repetitive work of typing each command in the terminal. This file containing a bunch of commands is known as a shell script and is saved with the extension of .sh.

Three types of looping statements can be used in shell scripting. These include:

1. **while statement**
   A while loop is a control flow structure that allows to repeatedly execute a block of code as

long as the condition specified is evaluated to be true

The syntax is:

while [condition]

do

#Statement blocks to be executed

done

Here, '[condition]' is the test condition to be evaluated before each iteration of the loop. If the condition is evaluated to be true, the statements in the body of the loop are executed. The iteration of the loop continues until the condition becomes false at which the control is transferred to the 'done' keyword signifying the termination of the while loop.

2. **for statement**

   A for loop is a control flow structure that allows repeatedly executing a block of code a specified number of times or for each item in a list.

   The syntax is:

   for i in list

   do

   #Statement blocks to be executed

   done

   Here, 'i' is the test iterator variable that takes the value of each item in the 'list'. The statements in the body of the loop are executed for each value of 'i'. The iteration of the loop continues until all the items in the list have been processed after which the control is transferred to the 'done' keyword signifying the termination of the for loop.

3. **until statement**

   An until statement is a control flow structure that allows repeatedly executing a block of code until a specified condition is true.

   The syntax is:

   until [condition]

   do

   #Statement blocks to be executed

   done

   Here, '[condition]' is the test condition that is evaluated before each iteration of the loop. If the condition is false, the statements inside the loop are executed. This process continues until the condition becomes true, at which point control is transferred to the statement following the done keyword.

   The until loop is similar to the while loop, except that the test condition is inverted. In other words, a while loop will continue to execute as long as the condition is true, while an until loop will continue to execute as long as the condition is false.

Note:

By default, the .sh file may not have executable permission. Hence, using the command chmod +x filename, the file is given executable permission.

#!/bin/bash signifies that the script should be executed using the bash shell

Algorithm:

For performing looping statements using shell scripting:

1. Take input from the user for the number of rows and store it in some variables
2. Use nested for loop to execute over the range of numbers till n, for rows and columns

3. Inside the nested for loop, write code that will print a * character for each column in the current row.
4. After the nested for loop for each row, write code to print a newline character to move to the next row.
5. The required output is generated

Program:

Steps to Execute the Program:

1. Create a new file and make it executable using the chmod command
2. Edit the file using Text Editor and add the commands required for performing the desired looping statements.
3. Save and close the file. The file extension should be .sh
4. Run the script in the terminal using ./filename.sh
5. Based on the operations, provide the necessary inputs and the output is generated based on the evaluation/iteration of specified looping statements.

Output Screenshots:

A.

E6_1.sh E6_2.sh

```
#!/bin/bash

echo "Enter the value of n:"
read n

for ((i=1; i<=n; i++))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "*"
    done
    echo
done

for ((i=n-1; i>=1; i--))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "*"
    done
    echo
done
```

Output:

```
student@Ubuntu:~/OS$ bash E6_1.sh
Enter the value of n:
5
*
**
***
****
*****
****
***
**
*
```

B.

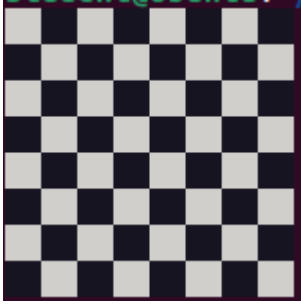
E6_1.sh | E6_2.sh

```
#!/bin/bash

for ((i=1; i<=8; i++))
do
    for ((j=1; j<=8; j++))
    do
        if [ $(( (i + j) % 2 )) -eq 0 ]
        then
            echo -e -n "\033[47m  "
        else
            echo -e -n "\033[40m  "
        fi
    done
    echo -e "\033[0m"
done
```

Output:

```
student@Ubuntu:~/OS$ bash E6_2.sh
```



C.

E6_3.sh

```
#!/bin/bash

day_no=1

for day in Mon Tue Wed Thu Fri Sat Sun
do
    if [ "$day" = "Sat" ] || [ "$day" = "Sun" ]
    then
        echo "Day $day_no: $day (weekend), wage = 550"
    else
        echo "Day $day_no: $day (weekday), wage = 350"
    fi

    day_no=$((day_no + 1))
done
```

Output:

```
student@Ubuntu:~/OS$ bash E6_3.sh
Day 1: Mon (weekday), wage = 350
Day 2: Tue (weekday), wage = 350
Day 3: Wed (weekday), wage = 350
Day 4: Thu (weekday), wage = 350
Day 5: Fri (weekday), wage = 350
Day 6: Sat (weekend), wage = 550
Day 7: Sun (weekend), wage = 550
```

D.

E6_4.sh

```
#!/bin/bash

echo "Enter a number:"
read n

fact=1
i=1

while [ $i -le $n ]
do
    fact=$((fact * i))
    i=$((i + 1))
done

echo "Factorial of $n = $fact"
```

Output:

```
student@Ubuntu:~/OS$ bash E6_4.sh
Enter a number:
5
Factorial of 5 = 120
```

E.

E6_5.sh

```
#!/bin/bash

echo "Enter n:"
read n

# Increasing pattern
for ((i=1; i<=n; i++))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "$j"
    done
    echo
done

# Decreasing pattern
for ((i=n-1; i>=1; i--))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "$j"
    done
    echo
done
```

Output:

```
student@Ubuntu:~/OS$ bash E6_5.sh
Enter n:
5
1
12
123
1234
12345
1234
123
12
1
```

F.

E6_6.sh

```
#!/bin/bash

echo "Enter n:"
read n

for ((i=1; i<=n; i++))
do

    for ((j=i; j<n; j++))
    do
        echo -n " "
    done

    for ((j=1; j<=i; j++))
    do
        echo -n "* "
    done

    echo
done

for ((i=n-1; i>=1; i--))
do

    for ((j=n; j>i; j--))
    do
        echo -n " "
    done

    for ((j=1; j<=i; j++))
    do
        echo -n "* "
    done

    echo
done
```

Output:



Conclusion:

These programs demonstrated the use of loops for printing patterns, displaying a chessboard, calculating weekday and weekend wages, and finding the factorial of a number. They helped in understanding the basic concepts of **loops, nested loops, conditional statements, arithmetic operations, and control structures in shell scripting**. Overall, the programs showed how shell scripts can be used to solve different problems efficiently and systematically.

Assessment Schemes:

Attendance	Discipline	Short oral	Correctness of Lab Report	Timely completion of Lab Report	Total marks (10)	Signature of Teacher with Date